

2025 겨울학기 컴퓨터네트워크 과제

국민대학교 시각디자인학과 20221625 정솔미

목차

1. [개요](#)
2. [실행 방법](#)
3. [동작 환경](#)
4. [HTTP 결과와 Wireshark 캡처](#)
 - 4.1. [POST](#)
 - 4.1.1. [201 Created](#)
 - 4.1.2. [404 Not Found](#)
 - 4.2. [GET](#)
 - 4.2.1. [200 OK](#)
 - 4.2.2. [404 Not Found](#)
 - 4.2.3. [400 Bad Request](#)
 - 4.3. [HEAD](#)
 - 4.3.1. [200 OK](#)
 - 4.3.2. [404 Not Found](#)
 - 4.3.3. [400 Bad Request](#)
 - 4.4. [PUT](#)
 - 4.4.1. [200 OK](#)
 - 4.4.2. [404 Not Found](#)
 - 4.4.3. [400 Bad Request](#)
 - 4.5. [???](#)
 - 4.5.1. [405 Method Not Allowed](#)
5. [소스 파일](#)
 - 5.1. [클라이언트\(c10.py\)](#)
 - 5.2. [서버\(s10.py\)](#)

1. 개요

Python 게시판 어플리케이션

목표

- TCP 기반 client, server 소켓 프로그램 작성 연습
- HTTP request, response message 구성 연습

기능

- HTTP method와 path를 입력해 header 구성, '작성자', '메시지' 필드에 입력해 body 구성
- db에 메시지 업로드 (POST)
- db에서 모든 메시지 또는 특정 인덱스의 메시지 조회 (GET)
- 특정 인덱스의 메시지 수정 (PUT)

2. 실행 방법

1. [Github](#)에서 소스 다운로드
2. 클라이언트 실행: `python3 c10.py`

만약 c10.py 실행 시 connection refused가 뜬다면

3. 로컬호스트용 서버 실행: `python3 s10-local.py`
4. 로컬호스트용 클라이언트 실행: `python3 c10-local.py`

생성된 창 ui를 통해 request 전송 가능

ui에서 response body, 터미널에서 request/response 전문 확인 가능

나머지 sn.py, cn.py는 과정을 남겨 둔 것이다. sn.py는 서버 코드, cn.py는 클라이언트 코드이다.
n=8까지는 로컬호스트를 사용했기 때문에 sn.py를 먼저 실행한 후에 cn.py를 실행해야 한다.

3. 동작 환경

서버(VPS)

- OS: Ubuntu 22.04 x64
- Python: 3.12
- Database: SQLite3
- Network: TCP socket

클라이언트(개인 노트북)

- OS: macOS
- Python: 3.12
- GUI: Tkinter
- Network: TCP socket

가상서버에서 s10.py를 실행하고 tmux를 사용하여 프로세스를 유지한다. 서버와 클라이언트는 가상서버의 공인 ip 주소를 통해 통신한다.

4. HTTP 결과와 Wireshark 캡처

4.1. POST

201 Created

정상적으로 db에 메시지를 업로드한 경우 생성된 엔트리의 값들을 반환한다.

ui 입력

An Array of Messages

header

HTTP method을 선택하세요

PUT/GET할 index를 입력하세요

☐ GET

☒ POST

☐ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

body

작성자 정솔미

메세지 안녕하세요

POST

ui 출력

An Array of Messages

header

HTTP method을 선택하세요

PUT/GET할 index를 입력하세요

☐ GET

☒ POST

☐ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

body

작성자 정솔미

메세지 안녕하세요

POST 201 Created

[4] '안녕하세요' - 정솔미

HTTP request, response message

```
POST / HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 29
```

정솔미 said:안녕하세요

```
HTTP/1.1 201 Created
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 33
```

[4] '안녕하세요' - 정솔미

Wireshark

The image shows a Wireshark packet capture window. The top toolbar indicates 'Capturing from utun4'. The packet list pane shows two packets:

No.	Time	Source	Destination	Protocol	Length	Info
5415	339.178187	10.0.5.183	158.247.200.157	HTTP	207	POST / HTTP/1.1
5417	339.194799	158.247.200.157	10.0.5.183	HTTP	190	HTTP/1.1 201 Created (text/plain)

The packet details pane for the selected packet (Frame 5415) shows the following structure:

- Frame 5415: Packet, 207 bytes on wire (1656 bits), 207 bytes captured
- Raw packet data
- Internet Protocol Version 4, Src: 10.0.5.183, Dst: 158.247.200.157
- Transmission Control Protocol, Src Port: 57603, Dst Port: 8080, Seq: 1
- Hypertext Transfer Protocol

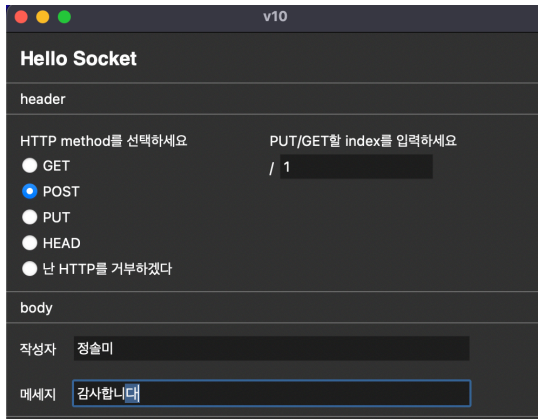
The packet bytes pane shows the raw data in hexadecimal and ASCII. The ASCII portion of the packet data is:

```
E.....@. ....
.....c&
.Vk POST / HTTP
1.1..Host: 158.
47.200.157..Use
-Agent: HelloSo
ket/1.0..Accept
text/pl ain; ch
rset=utf -8..Cor
ent-Leng th: 29
.....said
.....
```

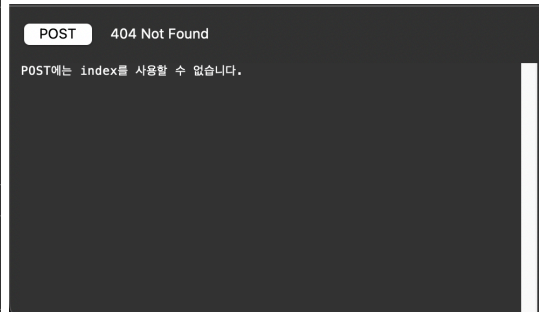
404 Not Found

POST에서 path가 '/'이 아닐 경우 안내문을 반환한다.

ui 입력



ui 출력



HTTP request, response message

```
POST /1 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 29
```

정솔미 said:감사합니다

```
HTTP/1.1 404 Not Found
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 47
```

POST에는 index를 사용할 수 없습니다.

Wireshark

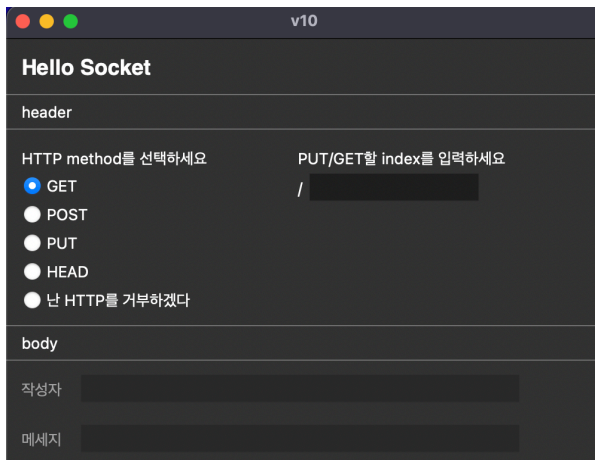
373	50.362798	10.0.5.183	158.247.200.157	HTTP	208	POST /1 HTTP/1.1
376	50.375666	158.247.200.157	10.0.5.183	HTTP	206	HTTP/1.1 404 Not Found (text/plain)

4.2. GET

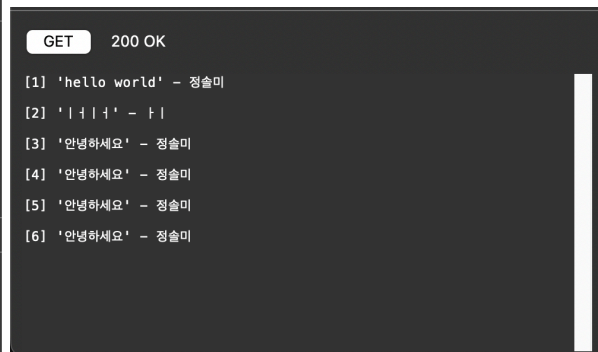
200 OK

Case 1: path가 '/'일 때
db의 모든 메시지를 반환한다.

ui 입력



ui 출력



HTTP request, response message

```
GET / HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 0
```

```
HTTP/1.1 200 OK
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 198
```

```
[1] 'hello world' - 정솔미
[2] '| | | |' - | |
[3] '안녕하세요' - 정솔미
[4] '안녕하세요' - 정솔미
[5] '안녕하세요' - 정솔미
[6] '안녕하세요' - 정솔미
```

Wireshark

14941	800.587700	10.0.5.183	158.247.200.157	HTTP	176	GET / HTTP/1.1
14944	800.603132	158.247.200.157	10.0.5.183	HTTP	351	HTTP/1.1 200 OK (text/plain)

Case 2: path가 '/n'일 때

인덱스 n의 메시지를 반환한다.

ui 입력

ui 출력

Hello Socket

header

HTTP method를 선택하세요

☒ GET
 ☐ POST
 ☐ PUT
 ☐ HEAD
 ☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

/ 1

body

작성자

메세지

GET 200 OK

[1] 'hello world' - 정솔미

HTTP request, response message

```
GET /1 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 0
```

```
HTTP/1.1 200 OK
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 29
```

```
[1] 'hello world' - 정솔미
```

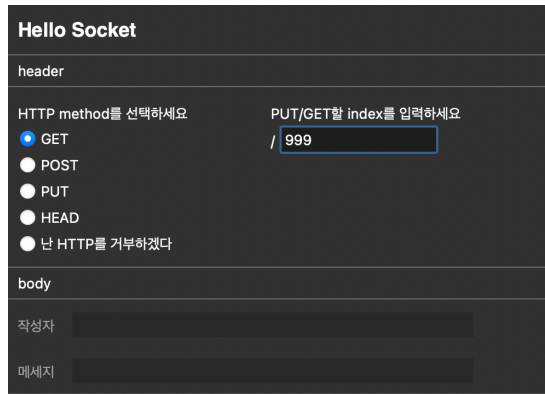
Wireshark

121	13.779241	10.0.5.183	158.247.200.157	HTTP	177	GET /1 HTTP/1.1
124	13.794821	158.247.200.157	10.0.5.183	HTTP	181	HTTP/1.1 200 OK (text/plain)

404 Not Found

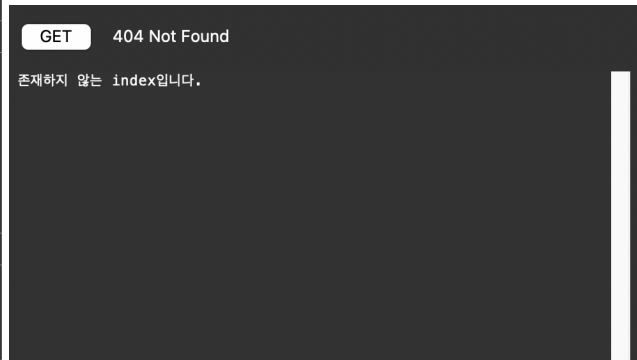
메시지가 존재하지 않는 인덱스를 입력한 경우 안내문을 반환한다.

ui 입력



The screenshot shows the 'Hello Socket' application interface. It has a 'header' section and a 'body' section. In the header, there are two labels: 'HTTP method를 선택하세요' and 'PUT/GET할 index를 입력하세요'. Under the first label, there are radio buttons for GET (selected), POST, PUT, HEAD, and a checkbox for '난 HTTP를 거부하겠다'. Under the second label, there is a text input field containing '/ 999'. The body section has two text input fields labeled '작성자' and '메세지'.

ui 출력



The screenshot shows the output of the application. At the top, there is a button labeled 'GET' and the text '404 Not Found'. Below this, the message '존재하지 않는 index입니다.' is displayed. The rest of the page is mostly empty with a dark background.

HTTP request, response message

```
GET /999 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 0
```

```
HTTP/1.1 404 Not Found
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 35
```

```
존재하지 않는 index입니다.
```

Wireshark

796	57.305633	10.0.5.183	158.247.200.157	HTTP	179	GET /999 HTTP/1.1
800	57.318771	158.247.200.157	10.0.5.183	HTTP	194	HTTP/1.1 404 Not Found (text/plain)

400 Bad Request

정수가 아닌 인덱스를 입력한 경우 안내문을 반환한다.

ui 입력

Hello Socket

header

HTTP method를 선택하세요

☒ GET
☐ POST
☐ PUT
☐ HEAD
☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

body

작성자

메세지

ui 출력

GET 400 Bad Request

index는 정수여야 합니다.

HTTP request, response message

```
GET /abc HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 0
```

```
HTTP/1.1 400 Bad Request
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 32
```

```
index는 정수여야 합니다.
```

Wireshark

3693	185.740310	10.0.5.183	158.247.200.157	HTTP	179	GET /abc HTTP/1.1
3695	185.754648	158.247.200.157	10.0.5.183	HTTP	193	HTTP/1.1 400 Bad Request (text/plain)

4.3. HEAD

HEAD는 GET과 동일하게 작동하지만 response body를 반환하지 않는다.

200 OK

ui 입력

ui 출력

Hello Socket

header

HTTP method를 선택하세요

☐ GET

☐ POST

☐ PUT

☒ HEAD

☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

/

body

작성자

메세지

HEAD

200 OK

HTTP request, response message

```
HEAD / HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 11
```

익명 said:

```
HTTP/1.1 200 OK
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
```

Wireshark

5970	314.840248	10.0.5.183	158.247.200.157	HTTP	189	HEAD / HTTP/1.1
5973	314.854983	158.247.200.157	10.0.5.183	HTTP	151	HTTP/1.1 200 OK

404 Not Found

ui 입력

Hello Socket

header

HTTP method를 선택하세요

☐ GET

☐ POST

☐ PUT

☒ HEAD

☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

/ 999

body

작성자

메세지

ui 출력

HEAD 404 Not Found

HTTP request, response message

```
HEAD /999 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 11
```

익명 said:

```
HTTP/1.1 404 Not Found
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
```

Wireshark

Time	Source	Destination	Protocol	Length	Info
10911	510.497338	10.0.5.183	158.247.200.157	HTTP	192 HEAD /999 HTTP/1.1
10913	510.510070	158.247.200.157	10.0.5.183	HTTP	158 HTTP/1.1 404 Not Found

400 Bad Request

ui 입력

Hello Socket

header

HTTP method를 선택하세요

☐ GET

☐ POST

☐ PUT

☒ HEAD

☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

/abc

body

작성자

메세지

ui 출력

HEAD 400 Bad Request

HTTP request, response message

```
HEAD /abc HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 11
```

익명 said:
□

```
HTTP/1.1 400 Bad Request
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 0
```

Wireshark

17955	606.759246	10.0.5.183	158.247.200.157	HTTP	192	HEAD /abc HTTP/1.1
17957	606.773565	158.247.200.157	10.0.5.183	HTTP	160	HTTP/1.1 400 Bad Request

4.4. PUT

200 OK

유효한 인덱스를 입력하면 새로 작성한 작성자, 메시지로 덮어쓰기하고 안내문을 반환한다.

ui 입력

Hello Socket

header

HTTP method를 선택하세요 PUT/GET할 index를 입력하세요

☐ GET / 3

☐ POST

☒ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

body

작성자

메세지 수정된 메세지입니다.

ui 출력

PUT 200 OK

성공적으로 3를 덮어쓰기했습니다.

```
[1] 'hello world' - 정솔미
[2] '||||' - |
[3] '수정된 메세지입니다.' - 익명
[4] '안녕하세요' - 정솔미
[5] '안녕하세요' - 정솔미
[6] '안녕하세요' - 정솔미
```

<= PUT 결과

HTTP request, response message

```
PUT /3 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 40
```

익명 said:수정된 메세지입니다.

```
HTTP/1.1 200 OK
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 46
```

성공적으로 3를 덮어쓰기했습니다.

Wireshark

29139	1081.326633	10.0.5.183	158.247.200.157	HTTP	218	PUT /3 HTTP/1.1
29141	1081.345612	158.247.200.157	10.0.5.183	HTTP	198	HTTP/1.1 200 OK (text/plain)

404 Not Found

메시지가 존재하지 않는 인덱스를 입력한 경우 안내문을 반환한다.

ui 입력

ui 출력

Hello Socket

header

HTTP method를 선택하세요

☐ GET

☐ POST

☒ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

PUT/GET할 index를 입력하세요

/ 999

body

작성자

메세지

메세지수정

PUT 404 Not Found

존재하지 않는 index입니다.

HTTP request, response message

```
PUT /999 HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 26
```

익명 said:메세지수정

```
HTTP/1.1 404 Not Found
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 35
```

존재하지 않는 index입니다.

Wireshark

33450	1313.159948	10.0.5.183	158.247.200.157	HTTP	206	PUT /999 HTTP/1.1
33452	1313.173207	158.247.200.157	10.0.5.183	HTTP	194	HTTP/1.1 404 Not Found (text/plain)

400 Bad Request

Case 1: 인덱스를 입력하지 않았을 때

PUT에서 인덱스를 입력하지 않았을 경우 안내문을 반환한다.

ui 입력

Hello Socket

header

HTTP method를 선택하세요 PUT/GET할 index를 입력하세요

☐ GET /

☐ POST

☒ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

body

작성자 정솔미

메세지 안녕하

ui 출력

PUT 400 Bad Request

뒤어쓰기할 index를 입력해 주세요.

HTTP request, response message

```
PUT / HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 23

정 솔 미 said:안 녕 하
```

```
HTTP/1.1 400 Bad Request
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 45

뒤 어 쓰 기 할 index를 입 력 해 주 세 요 .

```

Wireshark

186	11.494639	10.0.6.177	158.247.200.157	HTTP	184	PUT / HTTP/1.1
188	11.510323	158.247.200.157	10.0.6.177	HTTP	206	HTTP/1.1 400 Bad Request (text/plain)

Case 2: 인덱스가 정수가 아닐 때

정수가 아닌 인덱스를 입력한 경우 안내문을 반환한다.

ui 입력

Hello Socket

header

HTTP method를 선택하세요

PUT/GET할 index를 입력하세요

☐ GET

☐ POST

☒ PUT

☐ HEAD

☐ 난 HTTP를 거부하겠다

/ abc

body

작성자 정솔미

메세지 메세지수정

ui 출력

PUT

400 Bad Request

index는 정수여야 합니다.

HTTP request, response message

```
PUT /abc HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 29
```

정 솔 미 said:메 세 지 수 정

```
HTTP/1.1 400 Bad Request
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 32
```

index는 정 수 여 야 합 니 다 .

Wireshark

35702	1416.804527	10.0.5.183	158.247.200.157	HTTP	209	PUT /abc HTTP/1.1
35704	1416.820666	158.247.200.157	10.0.5.183	HTTP	193	HTTP/1.1 400 Bad Request (text/plain)

4.5. ???

405 Method Not Allowed

정의되지 않은 method를 사용한 경우 안내문을 반환한다.

ui 입력

ui 출력

Hello Socket

header

HTTP method를 선택하세요

PUT/GET할 index를 입력하세요

☐ GET

☐ POST

☐ PUT

☐ HEAD

☒ 난 HTTP를 거부하겠다

body

작성자

메세지

??? 405 Method Not Allowed

잘못된 method입니다.

HTTP request, response message

```
HTTP/1.1 405 Method Not Allowed
Server: MyHTTP/1.0
Content-Type: text/plain; charset=utf-8
Content-Length: 26
```

잘 못 된 method입 니 다 .

```
??? /jj HTTP/1.1
Host: 158.247.200.157
User-Agent: HelloSocket/1.0
Accept: text/plain; charset=utf-8
Content-Length: 11
```

익명 said:

□

Wireshark

19414	688.372887	158.247.200.157	10.0.5.183	HTTP	194	HTTP/1.1 405 Method Not Allowed (text/plain)
-------	------------	-----------------	------------	------	-----	--

5. 소스 파일

5.1. 클라이언트(c10.py)

클라이언트 프로그램은 Tkinter 기반 UI를 제공한다. UI의 전송 버튼을 누르면 `send_request` 함수가 호출된다. `send_request` 함수는 소켓을 생성하고, HTTP request를 서버로 전송하고, `response`를 받아 화면을 업데이트하는 역할을 한다.

Request 전송

클라이언트 소켓을 생성한다. `AF_INET`은 IPv4 주소 체계를 사용함을 의미하고, `SOCK_STREAM`은 TCP에서 사용하는 `stream` 방식의 소켓을 생성하도록 한다. 생성된 소켓은 서버의 ip 주소와 포트넘버로 `connect()`하여 서버와 연결된다.

```
serverName = '158.247.200.157'
serverPort = 8080
```

```
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
```

ui에서 `method`, `path`, 작성자, 메시지 입력을 가져와 가공한다. 작성자가 입력되지 않은 경우 기본값 '익명'을 사용한다. GET 요청이면 `body`를 빈 문자열로 하고, POST/PUT의 경우 프로그램 고유 포맷인 '작성자said:메시지' 형태로 `body`를 구성한다.

```
method = selected.get()
path = f"/{path_entry.get()}"
guest = entry_1.get() if entry_1.get() != "" else '익명'
guestbook = entry_2.get()
request_body = '' if method == "GET" else f"{guest}said:{guestbook}"
```

Request message를 구성한다. request line에는 `method`, `path`, 버전을 포함한다. `Host:` 는 요청을 보내는 서버의 주소를 나타내며, `User-Agent:` 는 임의의 클라이언트 이름을 지정했다. `Accept:` 는 클라이언트가 받을 수 있는 응답데이터의 타입을 의미하며 `text/plain`로 단순 텍스트임을 명시하고, `charset=utf-8`을 지정하여 한글 깨짐을 방지한다. `Content-Length:` 는 request body의 길이를 byte 단위로 나타낸다. `len("한글")`은 문자 수로 계산되어 2지만 `len("한글".encode())`는 byte 수로 계산되어 6으로 다르기 때문에 `.encode()`를 해야 한다. 빈 줄로 헤더의 끝을 알린 후 request body를 전송한다.

```

request_msg = (
    f"{method} {path} HTTP/1.1\r\n"
    f"Host: {serverName}\r\n"
    f"User-Agent: HelloSocket/1.0\r\n"
    f"Accept: text/plain; charset=utf-8\r\n"
    f"Content-Length: {str(len(request_body.encode()))}\r\n"
    f"\r\n"
    f"{request_body}"
)

```

request message를 인코딩하여 클라이언트 소켓을 통해 전송한다.

```

clientSocket.send(request_msg.encode())

```

Response 받기

클라이언트 소켓을 통해 서버로부터 response를 받아온다.

TCP는 데이터를 chunk로 나눠 보내기 때문에 한 번의 recv() 호출로 메시지가 전부 오지 않을 수 있다.

그래서 연속적으로 chunk를 받아 합친다.

TCP에서는 서버가 close()를 호출하면 FIN 패킷이 전송되고, 이후 클라이언트에서 recv()를 호출하면 빈 바이트열이 반환된다. 이를 기준으로 response 수신을 종료한다.

```

response = ""
while True:
    chunk = clientSocket.recv(1024).decode()
    if not chunk:
        break
    response += chunk

```

받아온 response를 빈 줄을 기준으로 header와 body로 분리한다. header를 줄바꿈을 기준으로 분리해 status line을 얻는다. status line을 띄어쓰기를 기준으로 분리해 status code와 status phrase를 얻는다.

```

header, body = response.split("\r\n\r\n", 1)
lines = header.split("\r\n")
version, status_code, status_phrase = lines[0].split(" ", 2)

```

파싱된 데이터를 가지고 ui를 업데이트한다.

```

status.config(text=f"{status_code} {status_phrase}")
response_body.delete("1.0", "end")
response_body.insert("end", body)
entry_1.delete(0, tk.END)
entry_2.delete(0, tk.END)
path_entry.delete(0, tk.END)

```

클라이언트 소켓을 닫는다.

```

clientSocket.close()

```

5.2. 서버(s10.py)

socket 서버 소켓을 생성한다. 파이썬 socket 인터페이스의 socket() 함수는 socket object를 리턴한다. AF_INET은 IPv4 주소 체계를 사용함을 의미하고, SOCK_STREAM은 TCP에서 사용하는 stream 방식의 소켓을 생성하도록 한다.

SOL_SOCKET은 해당 옵션이 프로토콜이 아니라 소켓 자체에 대한 옵션임을 알려준다.

SO_REUSEADDR는 TIME_WAIT 상태의 ip, port를 다시 bind할 수 있게 해준다. 서버를 종료하고 바로 같은 ip, port로 bind가 안되는 것이 번거로워서 넣었다.

bind() 함수는 소켓이 어떤 ip, port를 대표할지 등록한다. 0.0.0.0과 bind하여 모든 네트워크 인터페이스와 연결이 가능하게 했다. listen() 함수는 클라이언트 연결을 준비하도록 한다.

```

serverName = '0.0.0.0'
serverPort = 8080

serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
serverSocket.bind((serverName, serverPort))
serverSocket.listen(1)
print('the server is ready to receive')

```

db를 초기화한다.

```

conn = sqlite3.connect("messages.db")
cur = conn.cursor()
cur.execute("""

```

```

CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    guest TEXT NOT NULL,
    guestbook TEXT NOT NULL
)
"""
conn.commit()
conn.close()

```

db 함수들은 자체적으로 db와 연결을 열고 닫으며, 필요한 정보를 반환한다.

```

def add_message(guest, guestbook):
    conn = sqlite3.connect("messages.db")
    ...
    conn.close()
    return new_id

def get_all_messages():
    ...
    return rows

def get_one_message(msg_id):
    ...
    return row

def update_message(msg_id, guest, guestbook):
    ...
    return changed > 0

```

메인 루프

listen하고 있던 서버 소켓이 request를 받으면 커넥션 소켓이 생성된다. accept()는 연결 요청을 받으면 연결하고 새로운 소켓 객체와 주소를 리턴한다.

```

try:
    connectionSocket, addr = serverSocket.accept()
except Exception:

```

```
continue
```

소켓으로부터 데이터를 가져온다. `recv(1024)`는 한 번에 최대 1024 byte의 데이터를 소켓에서 받아온다는 것을 의미한다.

서버를 외부에 공개하자 다양한 스캐너나 공격이 왔다. 대부분 라우터, 프록시, Next.js 서버 등을 대상으로 한 요청으로 본 서버에는 의미 없는 요청이었다. 그러나 파싱 포맷에 맞지 않아 서버 프로그램이 종료되는 일이 많았다. 그래서 다양한 예외 처리를 추가하여 비정상 요청에도 종료되지 않게 하려고 했다.

`decode()`는 byte를 string으로 바꾼다. 비정상 요청 중 `gzip`, `multipart/form-data` 등 `text`가 아닌 것들이 있어 `utf-8`을 명시했고 `UnicodeDecodeError`면 파싱을 스킵하고 커넥션을 닫도록 했다. `data`가 없거나 줄바꿈이 없어도 파싱을 스킵하고 커넥션을 닫도록 했다. 한 번은 `ConnectionResetError`가 생겼는데 클라이언트 측에서 커넥션을 잘못 닫아서 발생하는 에러인 듯했다. 그래서 이 경우에도 파싱을 스킵하고 커넥션을 닫도록 했다. 그리고 커넥션을 너무 오랫동안 유지하는 방식의 `ddos` 공격도 있다고 하여 `timeout`도 넣었다.

```
connectionSocket.settimeout(5)

try:
    data = connectionSocket.recv(1024)
    if not data or b"\r\n\r\n" not in data:
        connectionSocket.close()
        continue
    try:
        request = data.decode("utf-8")
    except UnicodeDecodeError:
        connectionSocket.close()
        continue
except (ConnectionResetError, timeout):
    connectionSocket.close()
    continue

print(f"\r\n{request}")
```

받아온 request를 한 줄 띄움을 기준으로 header와 body로 분리한다. header를 줄바꿈을 기준으로 분리해 request line을 얻는다. request line을 띄어쓰기를 기준으로 분리해(포맷을 지키지 않았으면 연결을 닫는다) method와 path를 얻는다.

```
header, body = request.split("\r\n\r\n", 1)
lines = header.split("\r\n")
parts = lines[0].split()

if len(parts) != 3:
    connectionSocket.close()
    continue

method, path, version = parts
```

method와 path를 이용해 분기 처리한다. 'said:'로 작성자와 메시지를 분리하는 프로그램 고유 포맷을 지켰다면 split()으로 분리한다. db에 작성자와 메시지를 넣고 id를 받는다. id와 입력값으로 만든 string을 리턴한다. 201 Created는 주로 POST method에서 서버가 무언가를 성공적으로 생성했을 때 쓰인다. 본 프로그램에서는 '/'외의 path에 POST하는 기능이 없기 때문에 path가 '/'가 아닌 경우 404 Not Found를 돌려준다.

```
if method == 'POST':
    if not body or "said:" not in body:
        status_code = '400'
        status_phrase = 'Bad Request'
        response_body = "invalid body"

    elif path == '/':
        guest, guestbook = body.split("said:", 1)
        new_id = add_message(guest, guestbook)
        status_code = '201'
        status_phrase = 'Created'
        response_body = f"[{new_id}] '{guestbook}' - {guest}"
    else:
        status_code = '404'
        status_phrase = 'Not Found'
        response_body = "POST에는 index를 사용할 수 없습니다."
```


path에 메시지의 인덱스 정보를 넣어서 GET/PUT할 수 있도록 했다. 인덱스 없이 GET하면 db에서 모든 엔트리를 가져오는 함수를 호출하고, 인덱스가 정수면 그 메시지를 가져오는 함수를 호출한다.

그 인덱스의 메시지가 있어서 성공적으로 가져오면 200 OK를, 메시지가 없어서 가져오지 못하면 404 Not Found를 돌려준다. 200 OK는 클라이언트의 요청을 서버가 수신, 이해, 처리했음을 의미한다.

인덱스가 정수가 아니면 400 Bad Request를 돌려준다. 400 Bad Request는 대부분 클라이언트의 문제로(잘못된 url/cache/cookie) 서버가 요청을 이해하거나 처리하지 못할 때 사용한다.

```
elif method == 'GET':
    if path == '/':
        rows = get_all_messages()

        response_body = "\r\n".join(
            f"[{id}] '{guestbook}' - {guest}" for (id, guest, guestbook) in
rows
        )
    else:
        parts = path.strip("/").split("/")

        if parts[-1].isdigit():
            idx = int(parts[-1])
            row = get_one_message(idx)

            if row:
                id, guest, guestbook = row
                response_body = f"[{id}] '{guestbook}' - {guest}"
            else:
                status_code = "404"
                status_phrase = "Not Found"
                response_body = "존재하지 않는 index입니다."
        else:
            status_code = "400"
            status_phrase = "Bad Request"
            response_body = "index는 정수여야 합니다."
```

HEAD는 GET과 같지만 response body를 비워 둔다.

```

elif method == 'HEAD':
    parts = path.strip("/").split("/")

    if path == "/":
        response_body = ""
    elif parts[-1].isdigit():
        idx = int(parts[-1])
        row = get_one_message(idx)

        if not row:
            status_code = "404"
            status_phrase = "Not Found"
        else:
            status_code = "400"
            status_phrase = "Bad Request"

    response_body = ""

```

PUT과 올바른 인덱스를 입력하면 200 OK와 함께 해당 인덱스의 메시지를 수정해 주고, 수정되었다는 안내문을 보낸다. 인덱스를 입력하지 않으면 400 Bad Request로 url이 잘못되었음을 알려준다. 입력한 인덱스의 메시지가 없으면 404 Not Found를, 인덱스가 정수가 아니면 400 Bad Request를 돌려준다.

```

elif method == "PUT":
    parts = path.strip("/").split("/")

    if path == "/":
        status_code = "405"
        status_phrase = "Method Not Allowed"
        response_body = "덮어쓰기할 index를 입력해 주세요."
    elif parts[-1].isdigit():
        idx = int(parts[-1])

        guest, guestbook = body.split("said:", 1)
        guest = guest.strip()
        guestbook = guestbook.strip()

        ok = update_message(idx, guest, guestbook)

```

```

if ok:
    response_body = f"성공적으로 {idx}를 덮어쓰기했습니다."
else:
    status_code = "404"
    status_phrase = "Not Found"
    response_body = "존재하지 않는 index입니다."
else:
    status_code = "400"
    status_phrase = "Bad Request"
    response_body = "index는 정수여야 합니다."

```

405 Method Not Allowed는 주소와 method가 일치하지 않을 때 사용한다. 이 프로그램에서는 POST, GET, HEAD, PUT가 아닌 method를 보내면 405 Method Not Allowed를 돌려준다.

```

else:
    status_code = '405'
    status_phrase = 'Method Not Allowed'
    response_body = "잘못된 method입니다."

```

HTTP response message를 구성한다. status line에는 버전, status code, status phrase를 넣는다. Server:에는 임의의 서버프로그램 이름을 넣었다. Content-Type: 은 response body의 데이터 타입을 의미한다. Content-Length: 는 body의 길이를 byte 단위로 나타낸다. 빈 줄로 헤더의 끝을 알린 후 response body를 전송한다.

```

response = (
    f"{version} {status_code} {status_phrase}\r\n"
    f"Server: MyHTTP/1.0\r\n"
    f"Content-Type: text/plain; charset=utf-8\r\n"
    f"Content-Length: {str(len(response_body.encode()))}\r\n"
    f"\r\n"
    f"{response_body}"
)

```

response message를 encode하여 전송한다. sendall() 은 데이터 전체가 전송될 때까지 내부에서 반복되어 일부만 전송되는 문제를 방지하고자 했다.

```
try:
    connectionSocket.sendall(response.encode())
except ConnectionResetError:
    pass
```

커넥션 소켓을 닫는다.

```
connectionSocket.close()
```